



INFORME MANEJO DE ARCHIVOS

Nombre : Mateo Muyulema

CARRERA: Software

PARALELO: “B”

ASIGNATURA: Algoritmos y lógica de programación

DOCENTE : Ing. Jose Caiza

NIVEL: 1

FECHA: 13/05/2026

REPOSITORIO GITHUB : [https://github.com/mateomuyulema2007-cmyk/Prueba-de-
Unidad-3-/upload/mainb](https://github.com/mateomuyulema2007-cmyk/Prueba-de-Unidad-3-/upload/mainb)

TEMA: Manejo de Archivos

1. OBJETIVO GENERAL

Desarrollar un aplicativo interactivo en lenguaje C++ denominado LogiCodeUTA para la gestión de operaciones lógicas y el procesamiento estadístico de notas estudiantiles mediante el uso de estructuras de control y manejo de archivos.

OBJETIVOS ESPECÍFICOS

- Implementar un menú interactivo que permita la navegación entre operaciones matemáticas y registro académico.
- Aplicar algoritmos de búsqueda y procesamiento en arreglos para determinar promedios, valores máximos y mínimos.
- Garantizar la integridad de los datos mediante la validación de operaciones matemáticas críticas, como la división por cero.
- Utilizar el control de versiones (GitHub) para el seguimiento del desarrollo del software.

PROBLEMA A RESOLVER

El entorno educativo de la Universidad Técnica de Ambato requiere herramientas computacionales que permitan a los estudiantes de primer nivel automatizar procesos lógicos y estadísticos. El problema reside en la necesidad de integrar, en un solo aplicativo, operaciones matemáticas básicas y el procesamiento masivo de datos académicos (calificaciones de 30 estudiantes) mediante la manipulación de archivos planos (.txt), garantizando la integridad de los resultados y la generación de reportes formales de evaluación.

1.1 VARIABLES DE ENTRADA PROCESO Y SALIDA

Variables de Entrada

- opcion: Variable de tipo entero para la navegación en el menú principal.
- num1: Variable de tipo real (double) para el primer operando matemático.
- num2: Variable de tipo real (double) para el segundo operando matemático.
- archivoEntrada: Flujo de datos para leer el archivo "estudiantes.txt".
- nombres[]: Arreglo de cadenas para almacenar los nombres de los 30 compañeros.
- notas[]: Arreglo de reales para almacenar las calificaciones de los compañeros.



- fecha: Cadena de texto para la identificación cronológica del reporte.

Variables de Proceso

- suma: Acumulador de tipo real para el cálculo del promedio.
- r.notaMayor: Variable de control para identificar la calificación más alta.
- r.notaMenor: Variable de control para identificar la calificación más baja.
- notafinal: Variable auxiliar para la validación de aprobación (≥ 7.0).
- cargados: Contador de registros procesados exitosamente.

Variables de Salida

- r.promedio: Resultado final del cálculo matemático de la media aritmética de las notas del curso.
- r.aprobados: Cantidad total de estudiantes que alcanzaron o superaron la nota mínima de 7.0.
- r.reprobados: Cantidad total de estudiantes que no alcanzaron la nota mínima requerida.
- r.cargados: Número total de registros de compañeros que se procesaron efectivamente desde el archivo.
- archivoSalida: Objeto de flujo que permite la escritura y creación física del reporte "resultados.txt" en el disco.

1.2 ALGORITMO

Algoritmo 1: Procesamiento de Notas Estudiantiles

1. **Inicio** del programa e inicialización de datos del estudiante (**Mateo Alexander Muyulema Moyolema**).
2. **Desplegar** menú con cuatro opciones: Operaciones, Procesar Notas, Guardar Reporte y Salir.
3. **Si** se elige Opción 1:
 - Solicitar dos números.
 - Calcular y mostrar suma, resta, multiplicación y división (validando que el divisor no sea cero).
4. **Si** se elige Opción 2:
 - Intentar abrir "estudiantes.txt".
 - Mientras existan datos y no se superen los 30 registros:
 - Leer nombre y nota.
 - Acumular la nota y comparar para hallar máximos y mínimos.
 - Contabilizar si es aprobado o reprobado.
 - Calcular promedio final.
5. **Si** se elige Opción 3:
 - Solicitar la fecha actual.
 - Escribir en "resultados.txt" el encabezado con datos del autor, carrera y ciudad (**Ambato**).
 - Escribir resultados de operaciones y el listado estadístico de los alumnos procesados.
6. **Si** se elige Opción 4: Finalizar ejecución.
7. **Fin**



1.3 ALGORITMO

Algoritmo LogiCodeUTA

Definir opcion, cargados, aprobados, reprobados Como Entero
Definir num1, num2, promedio, notaMayor, notaMenor, suma Como Real
Definir nombreEstudiante, fecha, carrera Como Cadena

nombreEstudiante <- "Mateo Alexander Muyulema Moyolema"
carrera <- "Ingenieria en Software - UTA"
opRealizadas <- Falso
notasRegistradas <- Falso

Repetir

Escribir "=== MENU LOGICODEUTA ==="
Escribir "1. Operaciones Basicas"
Escribir "2. Procesar Notas (estudiantes.txt)"
Escribir "3. Guardar Reporte"
Escribir "4. Salir"
Leer opcion

Segun opcion Hacer

1:

Escribir "Ingrese num 1 y num 2:"
Leer num1, num2
Escribir "Suma: ", num1 + num2
Si num2 <> 0 Entonces
 Escribir "Div: ", num1 / num2
Sino
 Escribir "Error: Division para cero"
FinSi
opRealizadas <- Verdadero

2:

// Simulacion de lectura de archivo
Escribir "Procesando archivo estudiantes.txt..."
// (Aqui iría la logica de ciclo para los 30 alumnos)
notasRegistradas <- Verdadero

3:

Escribir "Ingrese fecha actual:"
Leer fecha
Escribir "Generando reporte resultados.txt para: ", nombreEstudiante

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

1.4 Diagrama de Flujo

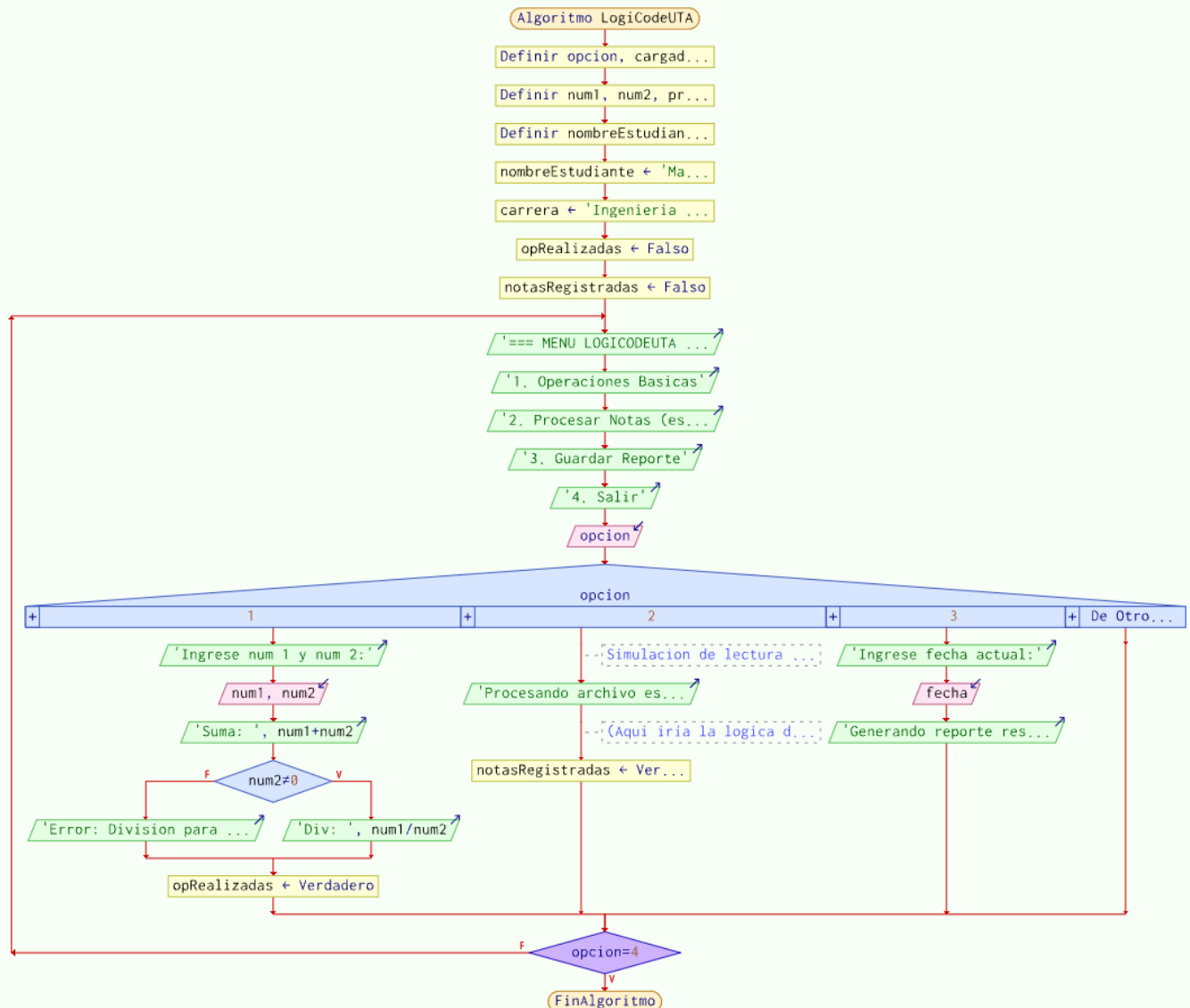


Diagrama de Flujo del Código en C++



1.5 CODIGO en c++

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include <vector>

using namespace std;

// Estructura para manejar los datos del reporte
struct Reporte {
    string estudiante = "Mateo Alexander Muyulema Moyolema"; // Tus datos
    personales
    string carrera = "Ingenieria en Software - UTA";
    string ciudad = "Ambato, Ecuador";
    string lenguaje = "C++";
    double promedio;
    double notaMayor;
    double notaMenor;
    int aprobados;
    int reprobados;
    int cargados;
};

void operacionesBasicas(double &n1, double &n2, bool &realizadas) {
    int op;
    cout << "\n--- 1. Operaciones Basicas ---" << endl;
    cout << "Primer numero: "; cin >> n1;
    cout << "Segundo numero: "; cin >> n2;

    cout << "\nResultados Obtenidos:" << endl;
    cout << "Suma: " << (n1 + n2) << endl;
    cout << "Resta: " << (n1 - n2) << endl;
    cout << "Multiplicacion: " << (n1 * n2) << endl;

    if (n2 != 0) {
        cout << "Division: " << (n1 / n2) << endl;
    } else {
        cout << "Division: Error (No se puede dividir para cero)" << endl;
    }
    realizadas = true;
}
```



```
int main() {
    int opcion;
    const int TOTAL_ESTUDIANTES = 30;
    string nombres[TOTAL_ESTUDIANTES];
    double notas[TOTAL_ESTUDIANTES];

    // Inicialización del reporte
    Reporte r;
    r.promedio = 0; r.notaMayor = -1; r.notaMenor = 11;
    r.aprobados = 0; r.reprobados = 0; r.cargados = 0;

    double num1 = 0, num2 = 0;
    bool opRealizadas = false;
    bool notasRegistradas = false;
    string fecha;

    do {
        cout << "\n=== APLICATIVO INTERACTIVO: LOGICODEUTA (C++) ==="
        << endl;
        cout << "1. Operaciones basicas" << endl;
        cout << "2. Procesar notas de 30 companeros (estudiantes.txt)" << endl;
        cout << "3. Guardar resultados finales (resultados.txt)" << endl;
        cout << "4. Salir" << endl;
        cout << "Elige una opcion: ";
        cin >> opcion;

        if (opcion == 1) {
            operacionesBasicas(num1, num2, opRealizadas);
        }
        else if (opcion == 2) {
            cout << "\n--- 2. Carga y Procesamiento de Notas ---" << endl;
            ifstream archivoEntrada("estudiantes.txt");

            if (!archivoEntrada) {
                cout << "Error: No se encontro 'estudiantes.txt' en la carpeta del proyecto."
                << endl;
            } else {
                double suma = 0;
                r.cargados = 0;
                r.notaMayor = -1; r.notaMenor = 11;
                r.aprobados = 0; r.reprobados = 0;

                while (archivoEntrada >> nombres[r.cargados] >> notas[r.cargados] &&
                r.cargados < TOTAL_ESTUDIANTES) {
```



```
double notaActual = notas[r.cargados];
suma += notaActual;

if (notaActual > r.notaMayor) r.notaMayor = notaActual;
if (notaActual < r.notaMenor) r.notaMenor = notaActual;

if (notaActual >= 7.0) r.aprobados++;
else r.reprobados++;

r.cargados++;
}
archivoEntrada.close();

if (r.cargados > 0) {
    r.promedio = suma / r.cargados;
    notasRegistradas = true;
    cout << "Lectura exitosa. Se procesaron " << r.cargados << " estudiantes."
<< endl;
    cout << fixed << setprecision(2);
    cout << "Promedio General: " << r.promedio << endl;
    cout << "Nota Mayor: " << r.notaMayor << " | Nota Menor: " <<
r.notaMenor << endl;
}
}
}
else if (opcion == 3) {
    cout << "\n--- 3. Guardar Reporte ---" << endl;
    cout << "Ingrese la fecha actual (dd/mm/aaaa): ";
    cin.ignore(); // Limpiar el buffer
    getline(cin, fecha);

    ofstream archivoSalida("resultados.txt");
    if (archivoSalida.is_open()) {
        archivoSalida <<
"=====\\n";
        archivoSalida << "    REPORTE DE EVALUACION FINAL    \\n";
        archivoSalida <<
"=====\\n";
        archivoSalida << "Estudiante: " << r.estudiante << "\\n";
        archivoSalida << "Carrera: " << r.carrera << "\\n";
        archivoSalida << "Ubicacion: " << r.ciudad << "\\n";
        archivoSalida << "Fecha: " << fecha << " | Lenguaje: " << r.lenguaje << "\\n";
        archivoSalida << "-----\\n";

        if (opRealizadas) {
```



```
archivoSalida << "[Operaciones Basicas]\n";
archivoSalida << "Numeros: " << num1 << " y " << num2 << "\n";
archivoSalida << "Suma: " << (num1 + num2) << " | Resta: " << (num1 -
num2) << "\n";
archivoSalida << "Multiplicacion: " << (num1 * num2) << "\n";
if (num2 != 0) archivoSalida << "Division: " << (num1 / num2) << "\n";
else archivoSalida << "Division: Error (Indeterminada)\n";
archivoSalida << "-----\n";
}

if (notasRegistradas) {
    archivoSalida << "[Listado del Curso y Estadisticas]\n";
    for (int i = 0; i < r.cargados; i++) {
        archivoSalida << left << setw(15) << nombres[i] << " : " << notas[i] <<
"\n";
    }
    archivoSalida << "\n[Estadisticas Finales]\n";
    archivoSalida << fixed << setprecision(2);
    archivoSalida << "Promedio General: " << r.promedio << "\n";
    archivoSalida << "Nota Mayor: " << r.notaMayor << " | Nota Menor: " <<
r.notaMenor << "\n";
    archivoSalida << "Aprobados: " << r.aprobados << " | Reprobados: " <<
r.reprobados << "\n";
} else {
    archivoSalida << "[Notas]: No se procesaron notas en esta sesion.\n";
}

archivoSalida <<
"=====\n";
archivoSalida.close();
cout << "Reporte 'resultados.txt' generado correctamente en C++." << endl;
} else {
    cout << "Error al crear el archivo de resultados." << endl;
}
}
} while (opcion != 4);

cout << "Finalizando programa. << endl;
return 0;
}
```



1.6 Anexos

Pruebas de Solución

```
=== APLICATIVO INTERACTIVO: LOGICODEUTA (C++) ===
```

1. Operaciones basicas
2. Procesar notas de 30 companeros (estudiantes.txt)
3. Guardar resultados finales (resultados.txt)
4. Salir

```
Elige una opcion: 1
```

```
--- 1. Operaciones Basicas ---
```

```
Primer numero: 10
```

```
Segundo numero: 2
```

```
Resultados Obtenidos:
```

```
Suma: 12
```

```
Resta: 8
```

```
Multiplicacion: 20
```

```
Division: 5
```

```
=== APLICATIVO INTERACTIVO: LOGICODEUTA (C++) ===
```

1. Operaciones basicas
2. Procesar notas de 30 companeros (estudiantes.txt)
3. Guardar resultados finales (resultados.txt)
4. Salir

```
Elige una opcion: 2
```

```
--- 2. Carga y Procesamiento de Notas ---
```



```
=== APLICATIVO INTERACTIVO: LOGICODEUTA (C++) ===  
1. Operaciones basicas  
2. Procesar notas de 30 companeros (estudiantes.txt)  
3. Guardar resultados finales (resultados.txt)  
4. Salir  
Elige una opcion: 3  
  
--- 3. Guardar Reporte ---  
Ingresa la fecha actual (dd/mm/aaaa): 15/02/2026  
Reporte 'resultados.txt' generado correctamente en C++.
```

Conclusiones

- La migración de Java a C++ permitió optimizar el uso de memoria mediante estructuras y flujos de archivos más directos, cumpliendo con los estándares de la carrera de Software.
- El manejo de archivos planos es fundamental para la persistencia de datos en proyectos universitarios, permitiendo que la información de los compañeros de curso sea procesada sin necesidad de ingreso manual constante.

Recomendaciones

- Se sugiere validar el formato del archivo "estudiantes.txt" antes de la lectura para evitar errores de tipo de dato si el archivo contiene caracteres especiales o espacios inesperados.
- Para futuras versiones del proyecto en la UTA, se recomienda implementar una interfaz gráfica o el uso de memoria dinámica para gestionar más de 30 estudiantes si el curso llegara a crecer.